1. List of Sample Programs: Provide at least five sample programs written in the language defined by the given grammar. These programs should demonstrate various constructs and operations permissible within the language. Make sure to make a few of the programs complex (multiple statements, etc.)

1. a = b + c
2. d = a - 1
3. a = 1 + 1 ; b = a - d
4. a = b + c ; c = a - d ; d = c + 1
5. a = b + c ; a = a + d

2. Explanation of Grammar Rules: Explain each grammar rule in detail, describing its purpose and the language constructs it represents. Discuss how these rules define the syntax of the language and constrain the structure of valid programs.

<program> → <stmts>

This serves as the entry point of the program and where compilation begins. It states that a program is simply a collection of statements and is constrained to the fact that a program must contain at least one stmt to be considered valid.

<stmts> → <stmt> | <stmt> ; <stmts>

This indicates that a collection of statements either consists of a single statement (base case) or a statement followed by a semicolon followed by another collection of statements (recursive case), which allows for chaining of numerous statements resulting in more complex programs. This serves as the sequence/recursion of instructions that are to be executed by a program. The syntax constrains the semicolon to only serve the purpose of chaining more than one statement together and does not serve as a terminator, meaning you must

only include a semicolon if you have a collection of statements following a prior statement in a recursive manner.

<stmt> → <var> = <expr>

This indicates that a single statement consists of a variable followed by an equal sign followed by an expression and serves as the assignment operator of the program. Programs following this grammar are constrained to this single action as statements can only be defined in this manner, meaning assignment of an expression solely to a variable is the only permissible action.

<var> → a | b | c | d

This defines the only characters permissible for declaring a variable. Meaning a program following this grammar may only assign an expression to characters “a” or “b” or “c” or “d”. This serves as the identifier space for the program meaning only 4 addresses in memory will be considered in the program, no more and no less, and can only be identified by four distinct variables.

<expr> → <term> + <term> | <term> - <term>

This indicates that an expression is limited to either a term followed by a plus sign (addition operator) followed by another term, or a term followed by a minus sign (subtraction operator) followed by another term. This serves as the program’s arithmetic logic, and that we can only assign the sum of or difference between two terms to a variable. We are constrained by the fact that expressions must adhere to only these two strict operations with no option for unary operators, chaining (expressions must consist of exactly two operands with a single operator in between), nor nested parentheses.

<term> → <var> | const

This indicates that a term can either be a variable or a constant. This serves as the data values of the program, which are to be used as the operands of expressions. It is constrained to the fact that terms can only be data values and cannot be expressions, meaning expressions themselves are limited to operation on exactly two values only, not on the results of other nested expressions.

3. Derivation: For each of the sample programs provided, perform a derivation using the given grammar. Show step-by-step how each program is derived from the starting symbol `<program>`. Include the sequence of productions applied at each step.

Program 1: a = b + c

Step	Current Form	Production Rule Applied
1.	<program>	<program> → <stmts>
2.	<stmts>	<stmts> → <stmt>
3.	<stmt>	<stmt> → <var> = <expr>
4.	<var> = <expr>	<var> → a
5.	a = <expr>	<expr> → <term> + <term>
6.	a = <term> + <term>	<term> → <var>
7.	a = <var> + <term>	<var> → b
8.	a = b + <term>	<term> → <var>
9.	a = b + <var>	<var> → c
10.	a = b + c	

Program 2: d = a - 1

Step	Current Form	Production Rule Applied
1.	<program>	<program> → <stmts>
2.	<stmts>	<stmts> → <stmt>
3.	<stmt>	<stmt> → <var> = <expr>
4.	<var> = <expr>	<var> → d
5.	d = <expr>	<expr> → <term> - <term>
6.	d = <term> - <term>	<term> → <var>
7.	d = <var> - <term>	<var> → a
8.	d = a - <term>	<term> → 1
9.	d = a - 1	

Program 3: a = 1 + 1 ; b = a - d

Step	Current Form	Production Rule Applied
1.	<program>	<program> → <stmts>
2.	<stmts>	<stmts> → <stmt> ; <stmts>
3.	<stmt> ; <stmts>	<stmt> → <var> = <expr>
4.	<var> = <expr> ; <stmts>	<var> → a
5.	a = <expr> ; <stmts>	<expr> → <term> + <term>
6.	a = <term> + <term> ; <stmts>	<term> → 1
7.	a = c + <term> ; <stmts>	<term> → 1
8.	a = 1 + 1 ; <stmts>	<stmts> → <stmt>
9.	a = 1 + 1 ; <stmt>	<stmt> → <var> = <expr>
10.	a = 1 + 1 ; <var> = <expr>	<var> → b
11.	a = 1 + 1 ; b = <expr>	<expr> → <term> - <term>
12.	a = 1 + 1 ; b = <term> - <term>	<term> → <var>
13.	a = 1 + 1 ; b = <var> - <term>	<var> → a
14.	a = 1 + 1 ; b = a - <term>	<term> → <var>
15.	a = 1 + 1 ; b = a - <var>	<var> → d
16.	a = 1 + 1 ; b = a - d	

Program 4: a = b + c ; c = a - d ; d = c + 1

1.	<program>	<program> → <stmts>
2.	<stmts>	<stmts> → <stmt> ;

		<stmts>
3.	<stmt> ; <stmts>	<stmt> → <var> = <expr>
4.	<var> = <expr> ; <stmts>	<var> → a
5.	a = <expr> ; <stmts>	<expr> → <term> + <term>
6.	a = <term> + <term> ; <stmts>	<term> → <var>
7.	a = <var> + <term> ; <stmts>	<var> → b
8.	a = b + <term> ; <stmts>	<term> → <var>
9.	a = b + <var> ; <stmts>	<var> → c
10.	a = b + c ; <stmts>	<stmts> → <stmt> ; <stmts>
11.	a = b + c ; <stmt> ; <stmts>	<stmt> → <var> = <expr>
12.	a = b + c ; <var> = <expr> ; <stmts>	<var> → c
13.	a = b + c ; c = <expr> ; <stmts>	<expr> → <term> - <term>
14.	a = b + c ; c = <term> - <term> ; <stmts>	<term> → <var>
15.	a = b + c ; c = <var> - <term> ; <stmts>	<var> → a
16.	a = b + c ; c = a - <term> ; <stmts>	<term> → <var>
17.	a = b + c ; c = a - <var> ; <stmts>	<var> → d
18.	a = b + c ; c = a - d ; <stmts>	<stmts> → <stmt>
19.	a = b + c ; c = a - d ; <stmt>	<stmt> → <var> = <expr>

20.	a = b + c ; c = a - d ; <var> = <expr>	<var> → d
21.	a = b + c ; c = a - d ; d = <expr>	<expr> → <term> + <term>
22.	a = b + c ; c = a - d ; d = <term> + <term>	<term> → <var>
23.	a = b + c ; c = a - d ; d = <var> + <term>	<var> → c
24.	a = b + c ; c = a - d ; d = c + <term>	<term> → 1
25.	a = b + c ; c = a - d ; d = c + 1	

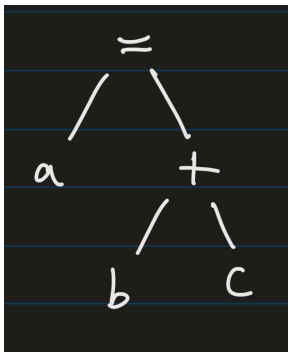
Program 5: a = b + c ; a = a + d

Step	Current Form	Production Rule Applied
1.	<program>	<program> → <stmts>
2.	<stmts>	<stmts> → <stmt> ; <stmts>
3.	<stmt> ; <stmts>	<stmt> → <var> = <expr>
4.	<var> = <expr> ; <stmts>	<var> → a
5.	a = <expr> ; <stmts>	<expr> → <term> + <term>
6.	a = <term> + <term> ; <stmts>	<term> → <var>
7.	a = <var> + <term> ; <stmts>	<var> → b
8.	a = b + <term> ; <stmts>	<term> → <var>
9.	a = b + <var> ; <stmts>	<var> → c
10.	a = b + c ; <stmts>	<stmts> → <stmt>
11.	a = b + c ; <stmt>	<stmt> → <var> = <expr>
12.	a = b + c ; <var> = <expr>	<var> → a

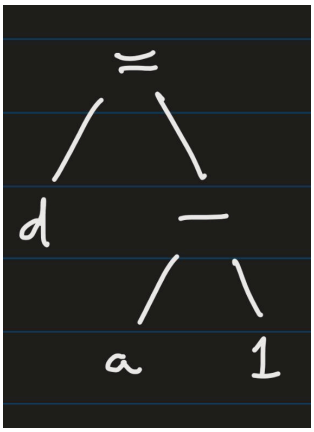
13.	a = b + c ; a = <expr>	<expr> → <term> + <term>
14.	a = b + c ; a = <term> + <term>	<term> → <var>
15.	a = b + c ; a = <var> + <term>	<var> → a
16.	a = b + c ; a = a + <term>	<term> → <var>
17.	a = b + c ; a = a + <var>	<var> → d
18.	a = b + c ; a = a + d	

4. Abstract Syntax Tree (AST): Construct an AST for each of the sample programs. Explain how the AST captures the hierarchical structure and semantics of the programs, emphasizing the relationship between nodes and the operations they represent.

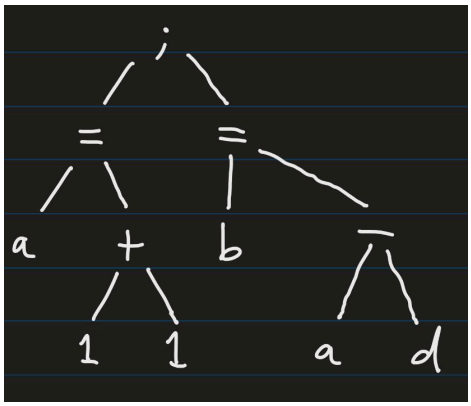
Program 1: a = b + c



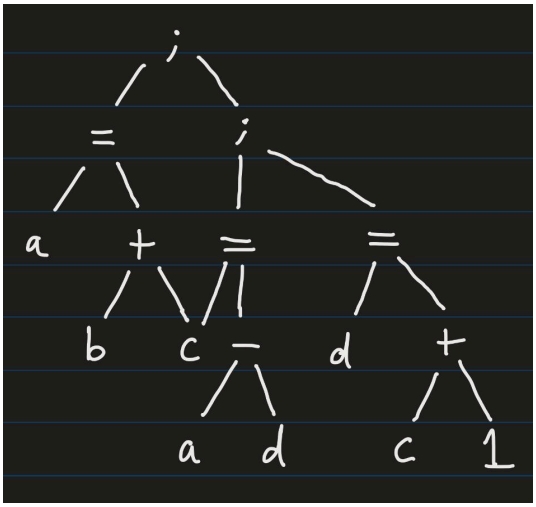
Program 2: d = a - 1



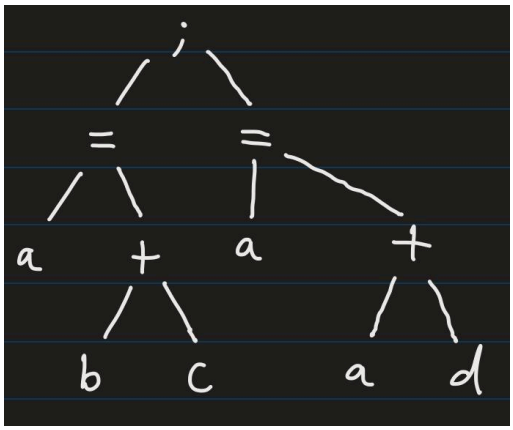
Program 3: a = 1 + 1 ; b = a - d



Program 4: a = b + c ; c = a - d ; d = c + 1



Program 5: a = b + c ; a = a + d



The AST captures the hierarchical structure by defining the order of precedence that the compiler must follow when translating the statements of the program. Leaves of the tree must consist of terms, either “a”, “b”, “c”, “d”, or a constant. Nodes must consist of operators, either “+”, “-”, or “=”, or “;” if conjoining two statements. Roots must consist of an “=” for a single statement, or a “;” if chaining together numerous statements. Starting at the root of the AST and traversing then writing each node's left child, then node, then right child, level by level, the AST enforces the semantics of the program through this data flow, i.e., for a = b + c, the compiler must first evaluate b + c and then assign the result of that to a via the “=” node.

5. Backwards Derivation: Reverse the process of derivation by starting from the ASTs obtained in the previous step and reconstructing the original source code for each program. Explain how this process relates to parsing and syntax analysis, highlighting the importance of the AST as an intermediate representation.

Program 1: a = b + c

1. Start at root =, traverse left child a, write a

2. Return to root =, write: =

3. Traverse right child, node +.

4. At +, traverse left child b, write: b

5. Return to +, write: +

6. Traverse right child c, write: c

7. Result: a = b + c

Program 2: d = a - 1

1. Start at root =, traverse left child d, write d

2. Return to root =, write: =

3. Traverse right child, node -.

4. At -, traverse left child a, write: a

5. Return to -, write: -

6. Traverse right child 1, write: 1

7. Result: d = a - 1

Program 3: a = 1 + 1 ; b = a - d

1. Start at root ;, traverse left child, node =

2. At =, traverse left child a, write: a

3. Return to =, write: =

4. Traverse right child, node +

5. At +, traverse left child 1, write: 1

6. Return to +, write: +

7. Traverse right child 1, write: 1

8. Subtree done, return to root ;, write: ;

9. Traverse right child, node =

10. At =, traverse left child b, write: b

11. Return to =, write: =

12. Traverse right child, node -

13. At -, traverse left child a, write: a

14. Return to -, write: -

15. Traverse right child d, write: d

16. Result: a = 1 + 1 ; b = a - d

Program 4: a = b + c ; c = a - d ; d = c + 1

1. Start at root ;, traverse left child, node =

2. At =, traverse left child a, write: a

3. Return to =, write: =

4. Traverse right child, node +

5. At +, traverse left child b, write: b

6. Return to +, write: +

7. Traverse right child c, write: c

8. Subtree done, return to root ;, write: ;

9. Traverse right child, node ;

10. At ; traverse left child, node =

11. At =, traverse left child c, write: c

12. Return to =, write: =

13. Traverse right child, node -

14. At -, traverse left child a, write: a

15. Return to -, write: -

16. Traverse right child d, write: d

17. Return to ;, write: ;

18. Traverse right child, node =

19. At =, traverse left child, node =

20. At =, traverse left child d, write: d

21. Return to =, write: =

22. Traverse right child, node +

23. At +, traverse left child c, write: c

24. Return to +, write: +

25. Traverse right child 1, write: 1

26. Result: a = b + c ; c = a - d ; d = c + 1

Program 5: a = b + c ; a = a + d

1. Start at root ;, traverse left child, node =

2. At =, traverse left child a, write: a

3. Return to =, write: =

4. Traverse right child, node +

5. At +, traverse left child b, write: b

6. Return to +, write: +

7. Traverse right child c, write: c

8. Subtree done, return to root ;, write: ;

9. Traverse right child, node =

10. At =, traverse left child a, write: a

11. Return to =, write: =

12. Traverse right child, node +

13. At +, traverse left child a, write: a

14. Return to +, write: +

15. Traverse right child d, write: d

16. Result: a = b + c ; a = a + d

The reverse process of derivation relates to parsing and syntax analysis, as it takes the source code of a program as input and breaks it down into individual statements, which are then broken down even further into individual terms (variables or constants) and expressions, which are once again broken down into operators and terms. By using these individual chunks or tokens of data, the compiler then organizes them into an AST to create an intermediate representation of the data in a hierarchical form, which is the actual parsing, so that the data can be assessed for accordance with the language syntax and, if it passes, can then be evaluated and executed in the correct order of precedence.